agentic PBT × 真实 CVE:全流程与对应代码

LLM agent 自动写性质测试找缺陷,但「找得好不好」原本只能靠人抽检。把每个项目退回修复前一刻,就有了标准答案。

阶段一 · 建数据集

让「有没有找到那个缺陷」变成可机械判定的问题

1 拉漏洞公告,粗筛

把 PyPI 全部已审核的漏洞公告(33MB 压缩包)一次拉下来,只留三样都满足的:有 CVE 编号、指向本仓库的修复提交、缺陷类别上性质测试有可能观测得到。

`dataset/build_dataset.py`

↓ `dataset.csv · 2942 行 / 694 个包`

2 定位被改的函数

取补丁的改动记录,算出它动了哪些行,再到打补丁之前的源码上做语法树解析,报出每一行所在的函数 —— 那才是 agent 将被指向的、还带着缺陷的代码。

`dataset/enrich_functions.py`

↓ `vuln_functions 列`

3 剔除平台假阴性

路径穿越类缺陷常常只在 Windows 上成立。扫描补丁里的平台相关写法,把这类样本标出来 —— 在 macOS 上跑它们,不管结果如何都说明不了问题。

`dataset/platform_scan.py`

↓ `platform 列 · 风险样本 17%`

4 把代码退回修复前一刻

克隆仓库,切到修复提交的父提交。这是整件事的支点:代码此刻一定还是坏的,于是「agent 有没有找到补丁修的那个缺陷」才有了标准答案。

`dataset/materialize.py`

17 个样本

每个 = 一份修复前的源码
+ 一个被指向的模块
+ 一条已知的标准答案(补丁修了什么)

指向修复前的代码 →

阶段二 · 跑 agent

每个样本重复 5 次

5 组装并发起一次运行

serial.sh 串行调度,一次只跑一个,撞上配额就等 10 分钟重试。go.sh 组装单次调用:把提示词拷成这次运行本地的斜杠命令、设好虚拟环境,再启动一个 agent。

`runs/serial.sh` `runs/go.sh`

↓ `claude -p "/hypo_v2 <模块名>"`

6 agent 照着剧本走 8 步 — `hypo_v2.md`

1. 看懂要测什么
   从参数里认出是模块、文件还是函数
2. 读实现
   导入它、读源码、看别处怎么调用它
3. 写下代码承诺了什么 → claims.md — 本次新增
   每条记五样:原话出处 / 来源种类 / 怎么从这话读出来的 / 规约本身 / 为什么不能读得更强
4. 规约 → 性质 — 本次新增
   只看 claims.md,不准回头重新判断一遍代码
5. 写测试 — 本次新增
   每个测试首行注释标明它对应哪一条规约
6. 跑 pytest,分诊失败
   区分真缺陷、测试写错、和本来就不该测的输入
7. 写报告
   复现脚本 + 为什么这是缺陷
8. 下结论
   找到了 / 没找到 / 这个目标没有可测的性质

↓ 每次运行留下四样东西

`claims.md`	代码承诺了什么,以及怎么读出来的
`test_*.py`	性质测试,每条标着它对应哪条规约
`bug_report_*.md`	找到缺陷时的报告与复现脚本
`claude_result.json`	这次运行的回合数与花费

为什么要把第 3 步单拎出来

「代码承诺了什么」和「我怎么检查它」原本挤在同一步。合在一起做,第一个判断错了,后面写出的测试会替它遮掩 —— 你只看到一个通过的测试,分不清是规约读错了,还是测试没碰到触发输入。这两种失败长得一模一样,改进方向却完全相反。拆开、并留下书面记录,事后才查得了。

运行产物 →

阶段三 · 量结果

把一堆运行产物变成可比较的数字

7 还原它提出过的每一条性质

只看报告是不够的 —— 报告里只有碰巧失败的那些。回放这次运行的完整记录,解析 claims.md,再用语法树拆开每个测试函数,得到:输入怎么生成、断言是什么、通过还是失败、背后是哪一条规约。通过的性质、和压根没想到的性质,才是解释「为什么没找到」的证据。

`dataset/extract_properties.py`

↓ 每条性质一行的表

8 判定这次有没有找到

三个各自独立的信号:有没有点名那个 CVE、有没有点名出问题的函数、有没有用补丁同样的说法描述缺陷。分开报而不是合成一个分数 —— 只中一个的多半是误判。命中仍需人工复核。

`dataset/classify_hit.py`

↓ 每次运行一行

9 5 次折叠成一个判断

单次运行几乎说明不了什么:同一条正确的性质在 litestar 上测得的命中率只有 33%,一次没找到,既可能是能力不行,也可能只是这次手气差。重复 5 次、报命中次数,才分得开这两件事。

`dataset/aggregate.py`

↓ 稳定命中 / 不稳定 / 从未命中

10 事前可算的筛选量

不跑也能算的指标,用来事先判断一个样本值不值得测。例如:被改的函数读了多少个自由变量、其中几个来自框架上下文 —— 那些是测试给不出来的值,给不出来就触发不了。

`dataset/check_i3.py`

最终产出

`table1_cve.csv`	样本本身:缺陷是什么
`table2_results.csv`	每条性质:提了什么、结果如何
`table3_stability.csv`	每个样本:5 次里命中几次

方框右下角 = 真正执行这一步的代码文件　　橙色标记 = 为「让规约可追溯」新增的步骤　　重画:/opt/miniconda3/bin/python docs/draw_pipeline.py